

Robot autónomo de aproximación y evasión basado en visión

Arquitectura del sistema: qué hace cada bloque y cómo se conectan

Federico Morán

Agosto de 2026

TPF — IA aplicada a la Robótica
Maestría en Inteligencia Artificial

Para qué son estas diapositivas

Objetivo

Poner en un solo lugar **qué estoy construyendo**, **por qué cada pieza está donde está**, y **cómo lo que falta se engancha con lo que ya funciona**.

Están organizadas en tres movimientos:

1. **El problema** — qué tiene que hacer el robot y por qué ese problema.
2. **La arquitectura** — el diagrama de bloques, y después una ficha por bloque: qué hace, cómo lo hace, con quién habla, en qué estado está.
3. **Los recorridos** — seguir un dato de punta a punta, y ver qué pasa cuando algo falla.

Al final: lo que ya está medido, lo que falta, y en qué orden.

1. El problema
2. La arquitectura: el diagrama de bloques
3. Bloque por bloque
4. Cómo interactúan: los recorridos
5. La máquina de estados en detalle
6. La ley de control
7. Lo que ya está medido
8. Estado y plan

El problema

El objetivo, en una frase

Un robot que **recorre una sala, reconoce dos objetos con la cámara y se comporta distinto con cada uno**: se acerca a uno y huye del otro.

Objetivo: teddy bear

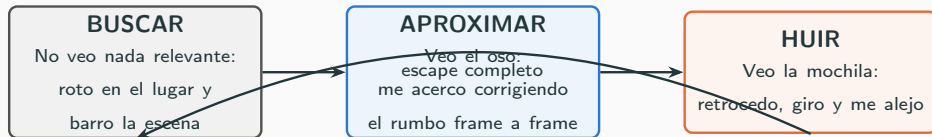
Lo busca, se aproxima controlando el rumbo con la cámara, frena al llegar y avisa con el buzzer.

Amenaza: backpack

Retrocede, gira 180° y se aleja. Tiene **prioridad incondicional** sobre todo lo demás.

Todo corre **a bordo**: no hay PC externa, ni teleoperación, ni nube.

Las tres conductas



Lo interesante no es cada conducta por separado

Es el **arbitraje**: qué pasa cuando el robot ve las dos cosas a la vez, y cómo se garantiza que la conducta de seguridad siempre gane.

Por qué este problema y no otro

Lo que el trabajo tiene que demostrar

Integrar **percepción con IA** en un **lazo de control físico** que corre en **tiempo real** sobre **hardware real**.

Este problema obliga a resolver, sí o sí, las cuatro cosas:

- **Percepción:** una red neuronal que corre a bordo, con un presupuesto de cómputo acotado.
- **Decisión:** una máquina de estados con *arbitraje de prioridades* — no alcanza con reaccionar, hay que resolver conflictos.
- **Control:** una ley que convierte píxeles en velocidades, con realimentación visual.
- **Actuación:** motores reales, con zonas muertas, asimetrías y caídas de tensión que ningún modelo ideal predice.

El punto 4 es el que da las sorpresas, y es donde está la mitad de las mediciones de este proyecto.

Lo que este trabajo deliberadamente NO hace

No se entrena un detector propio

Se usa YOLOv8n preentrenado en COCO y se filtran dos de sus 80 clases.

Por qué: el ciclo captura → etiquetado → entrenamiento → exportación agrega riesgo de cronograma sin tocar el núcleo del aporte, que es la **integración de inferencia en tiempo real en un lazo de control físico**.

Qué se gana a cambio: más profundidad en control, en arbitraje de conductas y en evaluación cuantitativa.

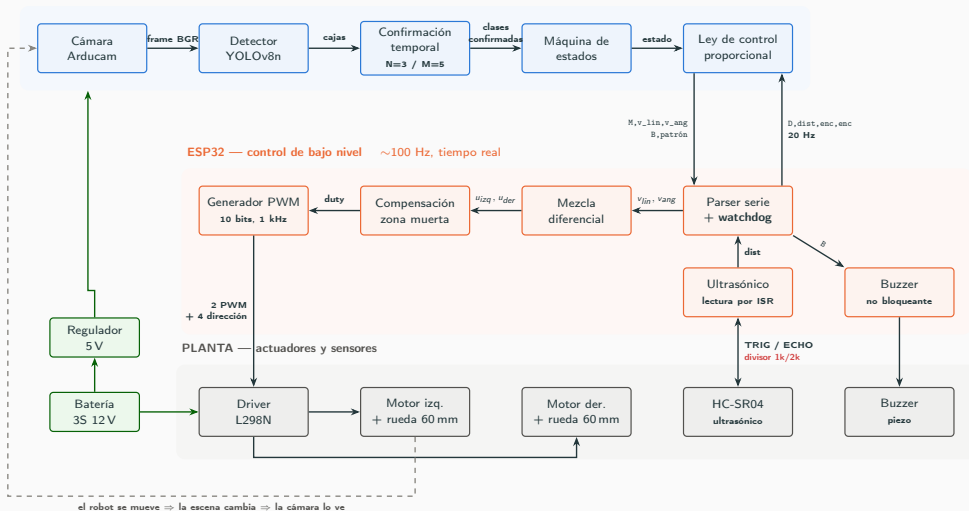
Criterio de elección de las clases

Objetos **medianos, compactos y con buen contraste**, detectables de forma estable a varios metros. Se descartaron book, remote, cell phone y scissors: chicos o planos, detección inestable a distancia con una cámara de bajo costo.

La arquitectura: el diagrama de bloques

El sistema completo

RASPBERRY PI 5 — percepción y decisión 5–15 Hz, no determinista



Cómo leer el diagrama

Tres dominios, tres colores

- **Azul** — lo que *piensa*: cámara, red neuronal, estados, control.
- **Naranja** — lo que *ejecuta en tiempo real*: parser, PWM, sensores.
- **Gris** — lo *físico*: driver, motores, sensor, buzzer.

Dos flujos que hay que distinguir

- **Hacia abajo**: la intención. De la imagen al par de tensiones que llegan a los motores.
- **Hacia arriba**: la realimentación. Telemetría del ESP32, y sobre todo el *lazo largo*: el robot se mueve, la escena cambia, la cámara lo ve.

La flecha punteada es la más importante de todo el diagrama

El lazo de control **no se cierra dentro del robot**: se cierra **a través del mundo físico**. No hay sensor que mida “estoy apuntando al oso”. Lo único que lo mide es el frame siguiente. Por eso el FPS de la cámara es la frecuencia del lazo de control.

El principio de diseño: dos niveles, dos frecuencias

Raspberry Pi 5 — alto nivel

- Linux, Python, red neuronal.
- **No determinista**: el sistema operativo puede robarle tiempo cuando quiera.
- 5–15 Hz, y con jitter.
- Decide *qué* hacer.

ESP32 — bajo nivel

- Bare metal, C, sin sistema operativo de por medio.
- **Determinista**: el PWM no se detiene nunca.
- ~100 Hz de lazo, 1 kHz de PWM.
- Decide *cómo* hacerlo.

La regla que hace que esto funcione

El Pi **nunca** toca un PWM. El ESP32 **nunca** toma una decisión de conducta.

Por qué importa: permite depurar cada nivel por separado — se puede probar el robot entero sin cámara (mandando comandos a mano) y se puede probar la visión entera sin robot. Es además la arquitectura estándar en robótica real: planificación contra control.

Bloque por bloque

Cómo está organizada esta sección

Una ficha por bloque, siempre con la misma estructura:

Qué hace

Su responsabilidad en una frase.

Con quién habla

Qué entra y qué sale, y en qué formato.

Cómo lo hace

El mecanismo concreto.

Estado

☒ listo ☐ en curso ☐ pendiente

El orden sigue el camino de la señal: del fotón que entra en la cámara al par que sale por el eje de la rueda.

Qué hace

Convertir la escena en una matriz de píxeles a ritmo constante.

Con quién habla

Sale: un array BGR de $480 \times 640 \times 3$ hacia el detector. **Entra:** luz. Y el movimiento del propio robot, que es lo que cierra el lazo.

Cómo lo hace

Módulo Arducam sobre la Raspberry Pi 5. Se captura a **640×480**: suficiente para detectar objetos medianos a 2–3 m y barato de mover. El código soporta las dos vías posibles —Picamera2 si es CSI, OpenCV/V4L2 si es USB— y elige sola.

Estado

■ Cámara disponible; falta la primera captura en el Pi.

Detalle que ya está resuelto en el código

El buffer de captura está forzado a **un** frame. Sin eso, OpenCV acumula imágenes viejas y el control termina reaccionando a una escena de hace medio segundo — un error clásico y difícil de diagnosticar después.

Qué hace

Encontrar en cada frame dónde están el oso y la mochila.

Con quién habla

Entra: frame BGR. **Sale:** lista de detecciones, cada una con clase, confianza y los dos números que necesita el control (*error_x* y *area_ratio*).

Cómo lo hace

Red neuronal convolucional preentrenada en COCO (80 clases). Se le pasa el frame y devuelve cajas con clase y confianza. Se filtran sólo las dos clases de interés: teddy bear y backpack.

Estado

■ Código escrito y verificado en PC. **Falta medir el FPS en el Pi** — es el número que destraba el resto del diseño.

Una decisión de ingeniería chica y útil

Las clases se declaran **por nombre** y los IDs de COCO se resuelven en el arranque contra `model.names`. Si algún día se cambia el modelo, el programa *falla ruidosamente al arrancar* en vez de detectar la clase equivocada en silencio. (Verificado: teddy bear=77, backpack=24.)

Qué hace

Evitar que un solo frame equivocado cambie la conducta del robot.

Con quién habla

Entra: el conjunto de clases detectadas en el frame actual. **Sale:** el conjunto de clases *confirmadas* hacia la máquina de estados.

Cómo lo hace

Un contador por clase: hacen falta **$N = 3$ frames consecutivos** para dar una clase por presente, y **$M = 5$ frames consecutivos** sin verla para darla por perdida. La asimetría es deliberada.

Estado

■ Implementado y verificado con casos de prueba.

Por qué N y M son distintos

Entrar es caro, salir es barato: confirmar rápido produce oscilaciones de conducta ante falsos positivos; olvidar rápido hace que el robot pierda el objetivo cada vez que una detección parpadea. Con $N=3$ / $M=5$ un falso positivo aislado no mueve nada, y una detección intermitente no hace abortar la aproximación.

Qué hace

Decidir *qué* conducta está activa en cada momento, y resolver conflictos entre conductas.

Con quién habla

Entra: clases confirmadas + distancia del ultrasónico. **Sale:** el estado activo, que determina qué ley de control se aplica.

Cómo lo hace

Cuatro estados — BUSCAR, APROXIMAR, HUIR, ENCONTRADO— con transiciones explícitas. La regla de arbitraje es fija: **HUIR gana siempre**, en cualquier estado y en cualquier momento.

Estado

☐ Día 4. Es el próximo bloque a escribir.

Por qué una máquina de estados y no un montón de condicionales

Porque el sistema tiene **memoria**: la conducta correcta depende de en qué estaba antes, no sólo de lo que ve ahora. Durante la fase de giro de la huida, el robot deja de ver la mochila — y sin embargo tiene que *seguir huyendo*. Un sistema puramente reactivo se detendría ahí.

Qué hace

Convertir la posición de una caja en la imagen en una intención de movimiento.

Con quién habla

Entra: la mejor detección de la clase relevante.

Sale: el par (v_{lin}, v_{ang}) , enteros en $[-100, 100]$, hacia el enlace serie.

Cómo lo hace

Control **proporcional** sobre dos errores independientes: cuán desviado está el objetivo del centro (gobierna el giro), y cuánto ocupa en la imagen (gobierna el avance). Dos ganancias, $K_{p,ang}$ y $K_{p,lin}$.

Estado

☐ Día 4. Los parámetros se calibran en piso en el Día 5.

Sin sensor de distancia real

No hay forma directa de saber a qué distancia está el oso. Se usa el **área de la caja como proxy**: crece al acercarse. Es aproximado —depende del tamaño real del objeto— pero es suficiente, y el ultrasónico da el criterio duro de llegada.

Qué hace

Transportar la intención hacia abajo y la telemetría hacia arriba.

Con quién habla

Pi → ESP32: M,<v_lin>,<v_ang> y B,<patrón>.

ESP32 → Pi:

D,<dist_cm>,<enc_izq>,<enc_der> a 20 Hz.

Cómo lo hace

ASCII delimitado por saltos de línea, 115200 baudios sobre USB. Legible por un humano en cualquier monitor serie, que es exactamente lo que se quiere para depurar.

Estado

■ Definido e implementado en el firmware, verificado en hardware.

Cuatro convenciones que evitan errores caros

- `dist_cm = -1` significa “no sé”, no “distancia cero”. Un `-1` leído como cero frena el robot sin motivo.
- Los encoders emiten 0,0 aunque no estén cableados: el parser del Pi es siempre el mismo.
- Las líneas que empiezan con # son diagnóstico para humanos, y el Pi las descarta.
- $v_{ang} > 0$ **gira a la derecha**, igual que $error_x > 0$ significa “objetivo a la derecha”: el mismo número sale del control y entra al protocolo *sin cambiar de signo*.

Qué hace

Recibir comandos, y **frenar el robot si el Pi deja de hablar**.

Con quién habla

Entra: bytes del puerto serie. **Sale:** el par (v_{lin}, v_{ang}) validado hacia la mezcla.

Cómo lo hace

Parser por líneas. En paralelo, un temporizador: si no llega ningún comando M durante **500 ms**, frena los dos motores — activo durante 200 ms y después suelta a rueda libre, para no dejar el puente en corto.

Estado

■ Implementado y **probado en hardware**.

Por qué el watchdog no es opcional

El Pi corre Linux, Python y una red neuronal: se puede colgar, el proceso se puede caer, el cable USB se puede soltar. **Un robot con los motores a fondo y sin cerebro sigue andando hasta chocar**. El watchdog es la única pieza del sistema cuyo trabajo es que las fallas de todo lo demás sean seguras.

Qué hace

Traducir “avanzar y girar” a “qué hace cada rueda”.

Con quién habla

Entra: (v_{lin} , v_{ang}). **Sale:** una magnitud con signo por rueda, normalizada.

Cómo lo hace

Un robot diferencial no tiene volante: gira porque las ruedas van a distinta velocidad. La mezcla es directa:

$$u_{izq} = v_{lin} + v_{ang} \quad u_{der} = v_{lin} - v_{ang}$$

Estado

■ Implementado y verificado.

La saturación se hace bien, y eso importa

Si la suma se pasa de 100, el firmware **escala el par completo** en vez de recortar cada rueda por separado. Recortar cambiaría la *relación* entre las ruedas — o sea, la curvatura pedida. El robot giraría distinto de lo que el control pidió, justo en las maniobras más agresivas, que son las que menos margen de error tienen.

Qué hace

Hacer que “mitad de comando” signifique de verdad “mitad de velocidad”, y lo mismo en las dos ruedas.

Con quién habla

Entra: u_{izq} , u_{der} normalizados. **Sale:** cuentas de duty de 10 bits para el generador de PWM.

Cómo lo hace

Cada rueda tiene su propia recta, con piso y techo propios:

$$d_i = \text{MIN}_i + (\text{TOP}_i - \text{MIN}_i) u$$

Más un **pulso de arranque** de 80 ms que rompe la fricción estática.

Estado

■ **Calibrado y verificado** el 27/08. Detalle en la sección de resultados.

Es el bloque que más trabajo llevó

Y el que más enseñó: los dos motores resultaron **no ser iguales**, y la zona muerta no estaba donde el modelo decía. Se desarrolla más adelante.

Qué hace

Convertir un número en una tensión media sobre el motor.

Con quién habla

Entra: duty en cuentas. **Sale:** 6 señales digitales al L298N: 2 de PWM (ENA, ENB) y 4 de dirección (IN1–IN4).

Cómo lo hace

Modulación por ancho de pulso a **1 kHz** con **10 bits** de resolución (0–1023), por el periférico LEDC del ESP32. Dos pines de dirección por motor definen el sentido de giro.

Estado

■ Verificado en hardware.

Por qué 10 bits y no los 8 habituales

Con 8 bits, la rueda izquierda quedaba con **24 cuentas útiles** entre su arranque y su tope: 24 velocidades distintas en todo su rango. No es un problema de física sino de *granularidad*. Pasar a 10 bits lo lleva a 96 sin tocar nada más. Después de calibrar quedaron **113 cuentas en la izquierda y 240 en la derecha**.

Qué hace

Amplificar señales lógicas de 3,3 V a la potencia que mueve los motores.

Con quién habla

Entra: 6 señales del ESP32 + 12 V de batería.

Sale: tensión conmutada a los dos motores.

Cómo lo hace

Doble puente H. El ESP32 no puede entregar ni de lejos la corriente de un motor; el L298 sí, alimentado directo desde la batería de 12 V.

Estado

■ Cableado y verificado en el bring-up.

La primera estimación que se cayó por medición

El datasheet hacía esperar una caída de tensión de **1,4 V**. Medida real: **2,5 V**. De los 12 V de la batería, al motor le llegan **9,5 V** a duty pleno.

Consecuencia: la velocidad máxima real es 0,87 m/s y no 1,1, y la zona muerta de abajo es más ancha de lo previsto. **Es tensión que se pierde en calor dentro del driver**, y es el argumento técnico para preferir un driver con MOSFET (TB6612, DRV8833) en un diseño futuro.

Qué hace

Convertir tensión en movimiento.

Con quién habla

Entra: tensión conmutada del L298N. **Sale:** movimiento — que es lo que cierra el lazo, a través de la cámara.

Cómo lo hace

Dos motorreductores GM25-370 de 12 V con ruedas de **60 mm**. Cada vuelta del eje de salida son **18,85 cm** de avance.

Estado

■ Caracterizados y calibrados.

La sorpresa más grande del proyecto

Los dos motores no son iguales. La rueda izquierda entrega **2,11 veces más velocidad por cuenta de duty** que la derecha, y además *cuesta más arrancarla*. Esa combinación no se explica por fricción: se explica por **relaciones de reducción distintas** (aproximadamente 2,1 : 1). De las dos unidades, sólo la derecha coincide con lo que dice la chapa del motor.

Qué hace

Medir la distancia al obstáculo que tiene enfrente — *vea o no vea* la cámara.

Con quién habla

Entra: eco acústico. **Sale:** `dist_cm` en la trama `D, ...`, o `-1` si no hay lectura válida.

Cómo lo hace

Emite un pulso de 40 kHz y mide el tiempo de vuelo del eco. El ESP32 lo lee **por interrupción**, sin bloquear: un pulso cada 60 ms, y el resultado viaja en la telemetría.

Estado

■ Sensor disponible; **falta cablearlo**. Pines `TRIG=18`, `ECHO=19`.

Riesgo real de hardware antes de conectarlo

El pin `ECHO` del HC-SR04 entrega **5 V** y el ESP32 **no tolera 5 V** en un GPIO. Va con **divisor resistivo 1 k Ω / 2 k Ω** . Sin el divisor se daña el pin de forma permanente. (`TRIG` sí acepta 3,3 V directo: su umbral de entrada es TTL.)

Por qué hay un ultrasónico si ya hay una cámara

Lo que la cámara no puede

- Ver un obstáculo que **no es una de las dos clases**: una pared, una pata de mesa, una caja.
- Dar una distancia **métrica**: el área de la caja es un proxy que depende del tamaño del objeto.
- Funcionar durante la fase de giro de la huida, cuando el objetivo está fuera del cuadro.

Lo que el ultrasónico aporta

- Un **criterio duro de llegada**, independiente de la visión.
- Seguridad anticolidión **siempre activa**, en todos los estados.
- 20 Hz de tasa de refresco, contra los 5–15 de la cámara.

Es fusión de sensores, en su forma más simple

Dos sensores con **modos de falla distintos**: la cámara falla con poca luz o con objetos parciales; el ultrasónico falla con superficies blandas u oblicuas. Ninguno cubre al otro del todo, pero juntos dejan muchos menos huecos que cualquiera solo. Y el ultrasónico cubre el caso más peligroso: *el obstáculo que el sistema no sabe nombrar*.

Qué hace

Hacer observable el estado interno del robot, sin mirar una pantalla.

Con quién habla

Entra: B,<patrón> desde el Pi (0 = apagado, 1 = hallazgo, 2 = alarma). **Sale:** sonido.

Cómo lo hace

Dos patrones distintos, generados de forma **no bloqueante** (el robot sigue controlando mientras suena): tono largo para hallazgo, intermitente para alarma.

Estado

■ Disponible, falta cablearlo. Pin 4.

No es un adorno

En la demo, el buzzer es la **única forma de saber qué está pensando el robot** sin cable ni monitor: si entró en ENCONTRADO, si detectó la amenaza, o si está buscando. Es instrumentación, y sirve tanto para depurar como para que se entienda el video.

A verificar: si el buzzer es **activo** en vez de pasivo, cambia una constante del firmware — el activo genera su propio tono y sólo hay que encenderlo.

Qué hace

Dar energía a dos consumidores con necesidades incompatibles.

Con quién habla

Entra: batería 3S LiPo. **Sale:** 12 V al L298N, 5 V al Pi.

Cómo lo hace

Dos rieles separados desde la misma batería: uno de 12 V directo a los motores, otro regulado a 5 V para la Raspberry Pi. **Las masas se unen en un solo punto.**

Estado

☐ Falta definir la fuente de 5 V para el Pi (buck de ≥ 3 A o power bank).

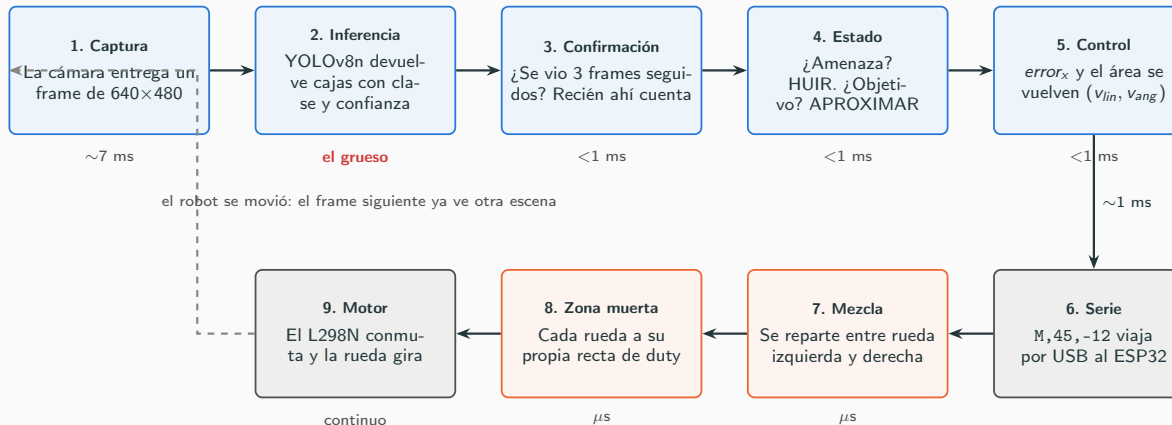
El riesgo marcado como “alta probabilidad” desde el día 1

Los motores tiran picos de corriente al arrancar y hunden la tensión del riel. Si el Pi cuelga de ese mismo riel, **se reinicia cada vez que el robot arranca** — y el síntoma parece un problema de software.

Por eso los rieles separados no son una mejora opcional: son requisito. Y la masa común tampoco: sin ella, las señales del ESP32 al driver no tienen referencia y el sistema hace cosas inexplicables.

Cómo interactúan: los recorridos

La vida de un frame: de la luz al par en la rueda



Los pasos 3 a 8 son *gratis* comparados con 1 y 2. **El costo del lazo está casi todo en la inferencia**, y por eso medirlo es el objetivo del Día 3.

Los tres lazos que corren en paralelo

Lazo de visión

5–15 Hz (a medir)

Cierra por el mundo físico.

Gobierna el rumbo y la conducta.

Si se cae: el watchdog frena.

Lazo de telemetría

20 Hz

Ultrasónico y encoders hacia el Pi.

Da el criterio duro de llegada y la seguridad anticolidión.

Si se cae: `dist = -1`, el Pi lo trata como “no sé”.

Lazo de seguridad

500 ms

El watchdog del ESP32.

No decide nada: sólo garantiza que el silencio signifique “parar”.

Si se cae: no hay red debajo. Por eso es lo más simple del sistema.

La jerarquía es deliberada

Cuanto más crítica la función, **más simple el código y más rápido el lazo**. La red neuronal es lo más complejo y lo más lento; el watchdog es veinte líneas de C y lo más rápido. Nada crítico depende de nada complejo.

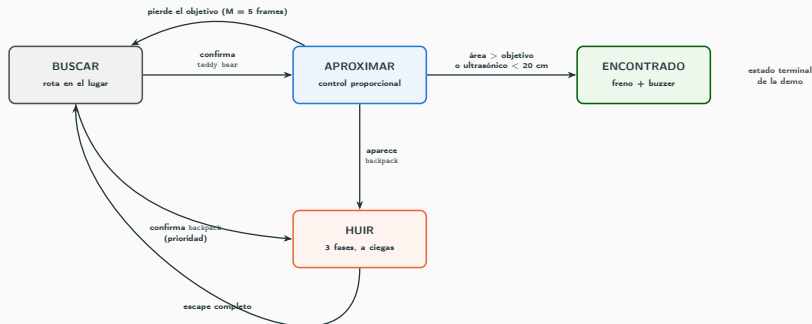
Qué pasa cuando algo falla

Falla	Qué se observa	Quién la cubre
El proceso de visión se cuelga, o se corta el cable USB	Dejan de llegar comandos M	Watchdog : frena a los 500 ms
La cámara devuelve un frame corrupto	Se descarta ese frame	El lazo de captura; el estado no cambia
Falso positivo aislado	Una detección espuria	Confirmación temporal (N = 3)
La detección parpadea	El objetivo aparece y desaparece	Confirmación temporal (M = 5)
Obstáculo que no es ninguna de las dos clases	La cámara no lo ve	Ultrasónico, siempre activo
El ultrasónico no lee	dist_cm = -1	El Pi lo trata como “no sé”, no como cero
La batería baja y el robot se desvía	Curva la trayectoria	La cámara lo corrige en APROXIMAR. En BUSCAR, no.
Una rueda se planta a comando bajo	El robot no arranca	Pulso de arranque + pisos calibrados

La última fila es la que ya **ocurrió de verdad** — y la que motivó toda la calibración.

La máquina de estados en detalle

Los cuatro estados



Todas las flechas hacia **HUIR** pueden dispararse *desde cualquier estado y en cualquier momento*. Es lo que la hace prioritaria.

BUSCAR — el estado sin información

Qué hace

Rota en el lugar a velocidad baja, barriendo la escena hasta que aparezca algo relevante.

Cómo sale

- → APROXIMAR si confirma el oso y no hay mochila.
- → HUIR si confirma la mochila (aunque también vea el oso).

Es el estado donde más duele la asimetría de los motores

Acá el robot gira **a ciegas**: no hay realimentación visual que corrija el rumbo, porque justamente todavía no vio nada. Si las ruedas no están parejas, el barrido se deforma.

Es uno de los dos únicos estados donde la calibración en lazo abierto tiene que aguantar sola.

Mejora prevista

Alternar rotación con pequeños avances, para no barrer siempre la misma escena si el objetivo está fuera del alcance visual.

APROXIMAR — el único estado con lazo cerrado

Qué hace

Se acerca al oso corrigiendo el rumbo **frame a frame** con la ley proporcional. El ultrasónico corre en paralelo por seguridad.

Cómo sale

- → ENCONTRADO si el área supera el objetivo o el ultrasónico baja de 20 cm.
- → BUSCAR si pierde la detección $M=5$ frames.
- → HUIR si aparece la mochila.

Acá la asimetría casi no importa

La cámara cierra el lazo de rumbo en cada frame, así que una trayectoria que se curva **se corrige sola**. El robot llega igual, describiendo un arco en vez de una recta.

Es la razón por la que **no** se agregaron encoders todavía: en el estado que más importa para la demo, la realimentación visual ya hace ese trabajo.

Dos criterios de llegada, no uno

Área o ultrasónico. Redundancia deliberada: si el oso es más chico o más grande de lo esperado, el área falla pero la distancia no.

HUIR — tres fases, y todas a ciegas

1. **Retroceso breve** ($\sim 1,5$ s), girando levemente en sentido opuesto a la mochila. Gana distancia antes de maniobrar.
2. **Giro en el lugar**, $\sim 180^\circ$. Acá la mochila *sale del cuadro*.
3. **Avance** durante unos segundos, con el ultrasónico activo para no chocar.

Al terminar las tres, vuelve a BUSCAR. Buzzer en patrón de alarma durante toda la maniobra.

La decisión de diseño clave

Durante las fases 2 y 3 el robot **no reevalúa la amenaza**. Es deliberado: si reevaluara, al perder de vista la mochila cancelaría la huida y volvería a girar hacia ella.

La maniobra se compromete y se termina.

Por eso hace falta una máquina de estados con memoria y no un sistema reactivo.

Dónde duele la asimetría

El giro de 180° y el avance final son en lazo abierto. Es el otro estado sin realimentación visual.

El arbitraje: por qué HUIR gana siempre

La regla

Ante detección simultánea de las dos clases, **la conducta evasiva tiene prioridad incondicional.**

Por qué es la regla defendible

En robótica, los comportamientos de **seguridad y preservación** siempre subsumen a los de objetivo. Es el principio de la *subsumption architecture* de Brooks: las capas de supervivencia pueden interrumpir a las de tarea, nunca al revés.

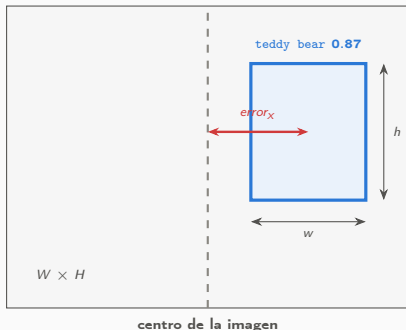
Por qué es verificable

Da un caso de prueba binario y sin ambigüedad: poner las dos cosas delante del robot y ver qué hace. La métrica correspondiente exige **10 de 10** corridas priorizando la huida — no 8 de 10 como el resto.

Sin una regla explícita, el comportamiento ante conflicto dependería del orden en que la red devuelve las cajas: **no determinista, y por lo tanto no evaluable.**

La ley de control

De una caja en la imagen a dos números



Error lateral — gobierna el giro

$$error_x = \frac{(x + w/2) - W/2}{W/2} \in [-1, +1]$$

Cero si el objetivo está centrado. Positivo si está a la derecha.

Área relativa — gobierna el avance

$$area_ratio = \frac{w \cdot h}{W \cdot H}$$

Crece al acercarse. Es el *proxy de distancia*.

Los dos son **adimensionales**: no dependen de la resolución de la cámara, así que cambiar de cámara no invalida las ganancias calibradas.

Giro

$$v_{ang} = K_{p,ang} \cdot error_x$$

Cuanto más desviado el objetivo, más fuerte el giro. Se anula solo cuando el objetivo queda centrado.

Avance

$$v_{lin} = K_{p,lin} \cdot (area_{obj} - area_ratio)$$

Avanza rápido cuando está lejos (área chica) y frena al acercarse. Se satura en $[0, v_{max}]$.

Por qué proporcional y no PID

- El **término integral** acumularía error mientras el robot está saturado o quieto, y produciría un tirón al reanudar (*windup*).
- El **término derivativo** amplificaría el ruido de la detección: la caja tiembla frame a frame aunque el objeto esté inmóvil.
- Con 5–15 Hz de realimentación y una planta lenta, el proporcional solo ya alcanza. **Agregar términos es agregar parámetros que calibrar y modos de falla que entender.**

La restricción que la calibración impuso al control

No hay arranque suave desde cero

Cualquier $u > 0$ produce al menos 0,091 m/s.

Es un **escalón**, no una rampa.

Consecuencia 1 — para parar

El Pi tiene que mandar **exactamente 0**. Un valor chico no hace que el robot vaya despacio: lo hace ir a 0,091 m/s.

Consecuencia 2 — zona muerta en el control

Hace falta una banda alrededor del centro donde $v_{ang} = 0$. Sin ella, un $error_x$ diminuto produce un giro de 0,09 m/s y el robot **oscila** alrededor del objetivo en vez de estabilizarse.

Y una corrección que apareció al escribir el código

El techo de velocidad documentado ($v_{max} = 0,34$) **quedó obsoleto** con la calibración: suponía que el duty escalaba desde cero. Con el mapa calibrado —lineal *en velocidad* entre 0,091 y 0,380 m/s— el valor correcto para 0,30 m/s es **72**, no 34. Con 34 el robot andaría a 0,19 m/s.

Los parámetros: qué está fijo y qué falta calibrar

Parámetro	Valor	Origen	Estado
Percepción			
Umbral de confianza	0,35	inicial	☐ calibrar contra falsos positivos
Tamaño de inferencia	640	inicial	☒ lo define la medición de FPS
N confirmar / M perder	3 / 5	diseño	☒
Control			
$K_{p,ang}$	0,8	inicial	☐ Día 5, en piso
$K_{p,lin}$	2,0	inicial	☐ Día 5, en piso
$area_{obj}$	0,15	inicial	☐ se mide con el preview
Zona muerta de $error_x$	0,08	diseño	☐
v_{lin} máximo	72	calculado	☒
Tracción			
PWM_MIN izq / der	125 / 112	medido	☒
PWM_TOP izq / der	238 / 352	medido	☒
PWM_BREAKAWAY	290 / 220	medido	☒
BREAKAWAY_MS	80	estimado	☐ ajustar en piso

Todos viven en un solo archivo, `src/config.py`, cada uno con su procedencia anotada.

Lo que ya está medido

El bloque de tracción está cerrado

Qué significa “cerrado”

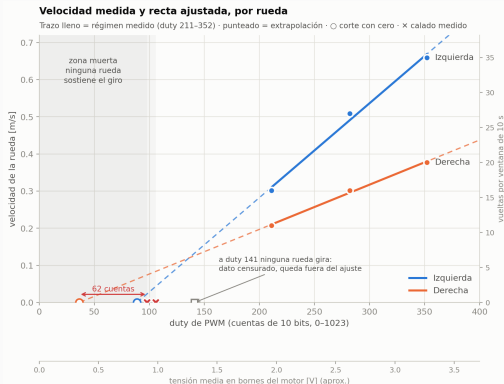
Firmware verificado en hardware, watchdog probado, motores caracterizados y calibrados, y **las dos ruedas girando parejas del 10 % al 100 % de comando**. Rango útil: **0,091 a 0,380 m/s**.

Llegar ahí llevó tres mediciones y **dos calibraciones fallidas**. Las próximas diapositivas resumen por qué, porque es la parte del trabajo con más contenido metodológico.

Todo el análisis es reproducible

Está en `scripts_auxiliares/01_calibracion_motores.ipynb`: los datos crudos embebidos, los ajustes recalculados desde cero y las figuras generadas por el propio cuaderno. Corre sin hardware.

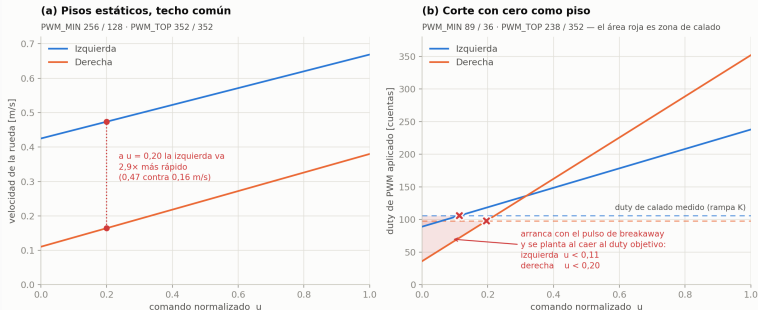
Hallazgo: los dos motores no son iguales



Lo que se esperaba: dos motores iguales con distinta fricción, o sea rectas paralelas con distinto arranque.

Lo que se midió: pendientes distintas. La izquierda da $2,11\times$ más velocidad por cuenta de duty. No es fricción: son reducciones distintas.

El error que la medición destapó



(a) Con los umbrales estáticos, a comando 0,20 una rueda iba $2,9\times$ más rápido que la otra. **(b)** Al corregirlo usando el *corte con cero* de la recta como piso, la rueda derecha quedaba **por debajo de su duty de calado** en toda la parte baja del rango: arrancaba con el pulso y se plantaba.

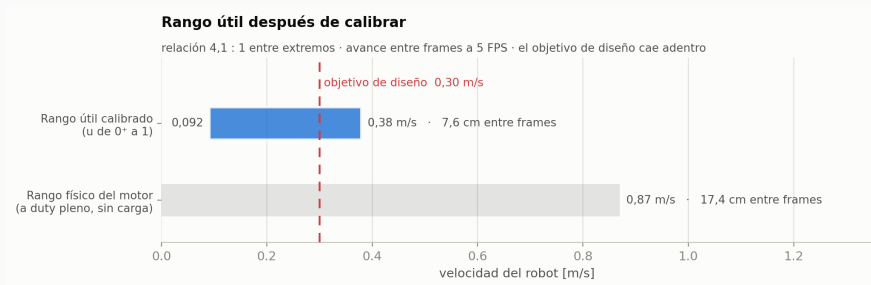
La lección metodológica — cuatro veces el mismo error

Estimación	Se predijo	Se midió	Dónde estaba la extrapolación
Caída del L298	1,4 V	2,5 V	Tabla del datasheet, tomada a otra corriente y temperatura
Simetría de los motores	iguales, distinta fricción	2,11× en pendiente	Se supuso desde la chapa, sin medir
Piso de la recta	89 / 36	~106 / ~98	Recta ajustada entre duty 211 y 352, usada en duty ≈ 0
Techo de velocidad	$v_{max} = 0,34$	72	Fórmula válida para el mapa anterior, aplicada al nuevo

El tercero es el más instructivo, porque el modelo era correcto

$R^2 > 0,99$ en el régimen medido. Lo que falló no fue el ajuste sino su dominio de validez: cerca de velocidad cero la fricción crece más que lineal, así que la recta describe muy bien lo que se midió y *no dice nada* sobre dónde se planta la rueda. El único remedio fue ir a medir ahí.

El resultado: un rango útil que entra en el presupuesto



La velocidad quedó acotada a un rango **4,1 : 1** que el lazo de visión puede seguir: entre 1,8 y 7,6 cm de avance entre frame y frame a 5 FPS.

Sin limitar, el robot iría a 0,87 m/s: **17 cm entre frames**, o sea controlando a ciegas la mayor parte del tiempo.

Estado y plan

Tablero de estado, bloque por bloque

Raspberry Pi — alto nivel

- ☐ Cámara Arducam (disponible)
- ☐ Detector YOLOv8n (código listo)
- ☒ Confirmación temporal
- ☐ Máquina de estados
- ☐ Ley de control

Energía

- ☒ Riel de 12 V a motores
- ☐ Riel de 5 V para el Pi

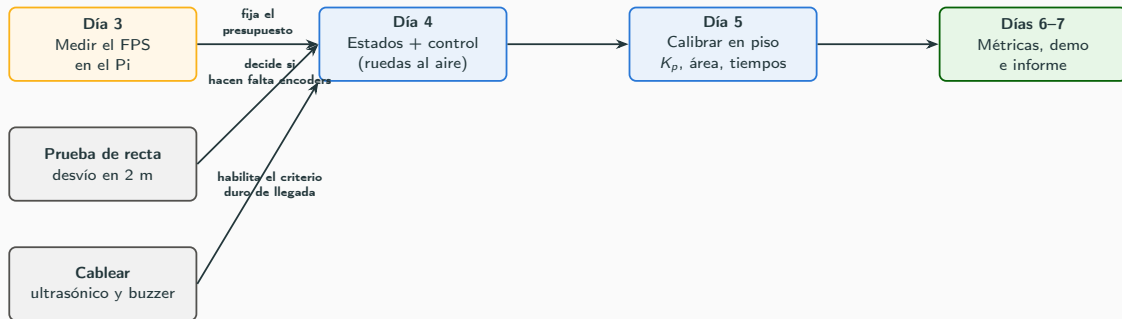
ESP32 — bajo nivel

- ☒ Protocolo y parser
- ☒ Watchdog de 500 ms
- ☒ Mezcla diferencial
- ☒ Compensación de zona muerta
- ☒ Generación de PWM
- ☐ Lectura del ultrasónico (sin cablear)
- ☐ Buzzer (sin cablear)

Planta

- ☒ L298N y motores, calibrados
- ☐ Prueba de recta en piso

Qué falta, y de qué depende cada cosa



Los tres pendientes de la izquierda son independientes entre sí

El **Día 3** sólo necesita el Pi y la cámara; los otros dos, el robot. Se pueden hacer en cualquier orden — pero el Día 3 es el que más destraba, porque el FPS medido es lo que fija el presupuesto de todo el lazo de control.

Riesgos vivos y cómo están mitigados

Riesgo	Prob.	Mitigación
FPS insuficiente en el Pi	Media	Bajar a 320×320 , exportar a NCNN, procesar 1 de cada 2 frames. <i>El banco de medición ya dice cuál corresponde.</i>
El Pi se reinicia al arrancar los motores	Alta	Alimentación separada desde el día 1. No es opcional.
Detección inestable a distancia	Media	Acercar el escenario, bajar el umbral de confianza, objetos más grandes.
El robot oscila al aproximarse	Media	Reducir $K_{p,ang}$ y agregar zona muerta alrededor del centro. <i>Sabemos que hace falta: el escalón de arranque lo garantiza.</i>
La marcha recta se curva	Confirmada	Calibración por rueda (hecha). Si no alcanza, encoders. <i>La prueba de recta es la que decide.</i>
Falla total de la detección	Baja	<i>Fallback:</i> marcadores ArUco. Sólo si todo lo demás falla, porque reduce el componente de IA.

Cómo se va a evaluar

Métrica	Cómo se mide	Objetivo
FPS de inferencia	Promedio del lazo en el Pi	≥ 5
Tasa de detección	% de frames con el objeto presente donde fue detectado	$\geq 80\%$ a 2 m
Falsos positivos	Detecciones espurias por minuto, sala vacía	$< 1/\text{min}$
Éxito de aproximación	Corridas que terminan en ENCONTRADO sin chocar	$\geq 8/10$
Éxito de escape	Corridas que completan la maniobra sin chocar	$\geq 8/10$
Arbitraje de conflicto	Corridas con ambos objetos donde prioriza huir	10/10
Tiempo hasta encontrar	Desde el inicio hasta ENCONTRADO, objeto a 2 m	registrar
Latencia de reacción	Desde que aparece la mochila hasta que arranca la huida	registrar

Los criterios están definidos **antes** de las pruebas. Definirlos después es ajustar la vara al resultado.

Resumen en tres frases

1. **La arquitectura es de dos niveles y esa es la decisión que ordena todo lo demás.** El Pi decide qué hacer a 5–15 Hz y sin garantías; el ESP32 ejecuta a 100 Hz con garantías. Ninguna función crítica depende de código complejo.
2. **El lazo de control se cierra por el mundo físico.** No hay sensor que mida “estoy apuntando al objetivo”: lo mide el frame siguiente. Por eso el FPS de la cámara es la frecuencia del lazo, y por eso medirlo es el próximo paso.
3. **La mitad del trabajo real estuvo en la planta, no en la IA.** Cuatro estimaciones razonables cayeron por medición, todas por el mismo motivo: extrapolar un modelo fuera del rango donde se lo validó. Es el hilo conductor del informe.

Próximo paso concreto: correr el banco de visión en el Pi y anotar el FPS.